

0.1 Abstract

This article introduces the reader to the g_m/I_D -based design methodology, which is a way to help CMOS analog circuit designers link physical transistor parameters to small signal models. It is written at the level of university students who are taking a first course on analog integrated circuits. It is also relevant to experienced engineers interested in a design flow that incorporates technology details early in the design cycle and yields excellent agreement between hand-calculations and circuit simulations.

0.2 Introduction

Following perhaps a long road to maturity, CMOS has become an excellent platform for analog circuit design. Not only is it unrivaled in switching and charge-mode processing, but it benefits from persistent process improvements fueled by the digital consumer market. Unfortunately, designers may find it very difficult to take advantage of these strengths. A primary reason for this is that CMOS behavior is hard to predict without using very complex models, and this complexity only worsens with technology scaling. Designers, incidentally under pressure to meet deadlines, are forced to either incorporate complex models into their hand calculations or spiral into a Spice-intensive design loop. Neither of these strategies are as effective or pleasant as we would like.

My goal in this article is to introduce you to the g_m/I_D -based design methodology, which greatly improves the predictability of CMOS small-signal behavior without requiring complex equations. We will define the ratio g_m/I_D in more detail later, but for now, just think of it as a design variable that encapsulates the biasing conditions of a MOS transistor. Or, even more concisely:

$$g_m/I_D \approx \text{bias point} \rightarrow \text{small-signal model}$$

Development of the methodology will involve several steps. We will start with a very broad overview of analog circuit design to see what problem it is that we are attempting to solve, and how it has been solved in the past. I will then explain, at a qualitative level,

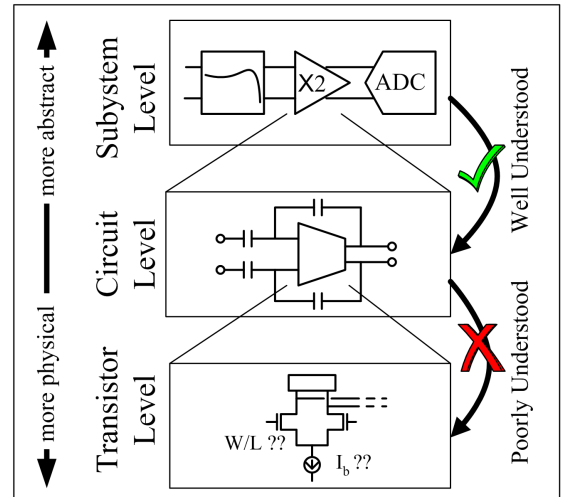


Figure 1: Low-level circuit implementation is often more difficult than higher-level design.

why the g_m/I_D -based approach is the best tool for solving this problem. Next, we will rehash the entire discussion at a quantitative level. This will entail a review of transistor operation and a chronological development of the tools we have available. Finally, we will close with a thorough design example.

0.3 The Big Picture

0.3.1 Analog Design Relies on Abstraction

Fig. 1 shows several levels of abstraction in which we can view an analog design. Thanks to abstractions, engineers working at the higher levels can perform analysis using linear Signals-and-Systems theory. This is the domain of filters, gain blocks, OpAmp circuits, etc. The mathematics that govern this realm are elegant, often with centuries-old roots. Consequently, we have gotten very good at understanding how to work with these blocks. Most engineering schools send students through an entire battery of courses that satisfactorily cover this area.

Descending to the lower levels, there is a different story. While we find it straightforward to build a gain-of-two stage using an OpAmp, we find it very difficult to build the OpAmp itself. How big should each transistor be? How much bias current is needed? These low-level decisions can be unclear, and there are two big reasons why. First, transistor behavior

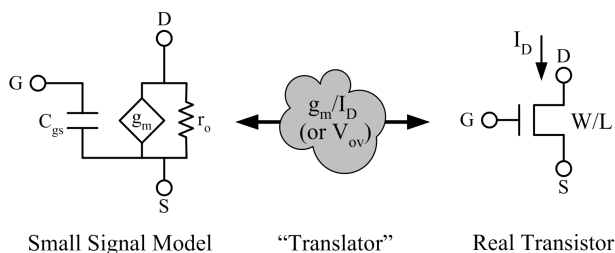


Figure 2: Small-signal model of a transistor.

is nonlinear, and classical Signals-and-Systems analyses fall apart when applied to nonlinear systems. Second, technology advancements change the rules of the game faster than we can make them. There are no centuries-old roots here! As a result, we simply do not have a nice set of transistor equations that is both compact enough for hand calculations and accurate enough to match Spice simulations.

0.3.2 Making Low-Level Design Manageable

We can make low-level design easier if we transform transistors into Signals-and-Systems-friendly devices. As you know, we do this by approximating each transistor with a few ideal elements, collectively referred to as a *small-signal model*. Fig. 2 shows a basic and familiar small signal model of a MOSFET. It also highlights the translational role that g_m/I_D (or its predecessor, V_{ov} , another biasing variable) plays in the design process. Of course, the drawback of using small-signal models is that they introduce errors, as all approximations must. But that is far outweighed by the benefits of using Signals-and-Systems techniques, without which we would not have concepts like gain, bandwidth, frequency response, poles, and zeros!

Fig. 3, then, is a good illustration of how the g_m/I_D -based design methodology fits into the big picture. At the top is the abstract Signals-and-Systems world, where we are very comfortable. At the bottom are physical transistors, which, in the end, must behave the way we want them to. Sitting in the middle of all this is g_m/I_D , an intermediate biasing variable that bridges the abstract-to-physical gap very well. Keep this picture in mind as we continue our discussion.

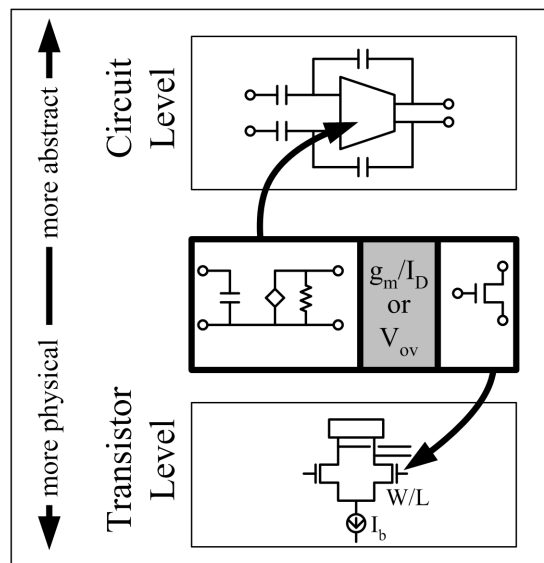


Figure 3: Small signal models allow us to use transistors in a Signals-and-Systems context.

0.3.3 Why g_m/I_D is Better than V_{ov}

V_{ov} -based design, which we will shortly cover in more detail, long predates g_m/I_D -based design. As we have already hinted, both V_{ov} and g_m/I_D are quantities that tell you something about the bias point of a transistor. So, how are these approaches different?

When CMOS designers choose to follow a V_{ov} -based design strategy, they implicitly accept the validity of the *long-channel* model. I am certain that you are familiar with the long-channel model (we will also review it in a later section). When we were first taught how to analyze a MOSFET, we were shown a derivation of it using basic calculus. Unfortunately, most of the assumptions that make the derivation so clean are untrue for today's small geometries. Consequently, the V_{ov} -based methodology no longer yields circuits that behave as intended. In order to salvage the model, designers have tried to patch it with short-channel effects and a variety of curve-fitting terms that are (sometimes only wishfully) based on different physical arguments. But in the end, V_{ov} -based design only gets harder and less accurate.

Our new strategy, g_m/I_D -based design, does not rely on the validity of the long-channel model. In fact, it does not rely on the validity of anything except simulation. This methodology is lookup-table-based. The underlying philosophy is that the equations governing MOSFETs are so complex that we must get

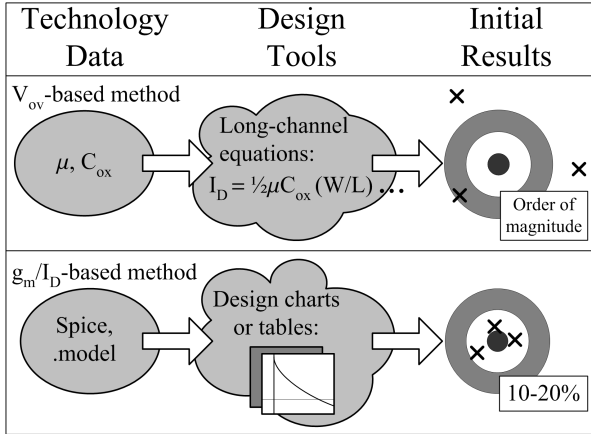


Figure 4: High-level comparison of two popular design methodologies.

rid of them in favor of a few tables or graphs. And because these graphs are generated using device simulations in Spice, they are much more accurate than the long-channel model could ever hope to be.

It is no stretch to say that most of us cringe a little at the thought of using a lookup table. With the advent of cheap and powerful computing, we electrical engineers have lost touch with filter tables, log tables, trigonometric tables, and the like. But our current equational exclusivity has been only a brief fad in our industry. Just as vacuum tube circuit designers once used (and still use!) tube curves, so we are rediscovering the value of table-based design for situations where it is the most efficient means of "computation."

Fig. 4 compares V_{ov} -based design and g_m/I_D -based design in a side-by-side summary. In both cases, we need physical information about the technology target. After all, the capabilities of the target will obviously affect transistor performance greatly. In the case of the long-channel model, the technology data must be limited to only the barest of essentials, such as μ and C_{ox} , otherwise hand calculations become intractable. Consequently, initial designs may only get within an order of magnitude until the designer gets a "feel" for that process. Meanwhile, the g_m/I_D -based method utilizes complete Spice models from the technology target and yields initial results that only require minor tweaking.

0.4 The More-Detailed Picture

Now I want us to start over trying to solve the design problem, but at a more quantitative level. We will reach the same conclusion, of course, even though we are taking a very different approach.

A First Attempt at Transistor-Level Design

How might an intelligent-but-inexperienced engineer go about designing a circuit? Of course, I have a preferred method towards which I am working, but it is certainly worthwhile to see if we can solve the design problem without knowing the answer ahead of time.

To begin, let us step back and ask, what will our finished design look like? Or, what *is* a finished design? In the context of this article, it is a netlist. Ultimately we just want a file that contains specifications for all the transistors, resistors, capacitors, etc., and explains how they are all connected together. Of course, in the real world, circuits must be fabricated, and designers must be wary of the limitations of simulation itself, and how well it agrees with actual measured performance, but those concerns are beyond our scope here.

If our end goal is a netlist, why not start with the netlist and work backwards? What kinds of information do we need in order to "fill in the blanks?" For reference, here is a line that instantiates a transistor in Hspice:

```
M1 drn gat src blk nchmodel L=0.18u W=10u
```

Well, which blanks can we fill in? Put another way, how do we *design* a transistor? Obviously, V_T , μ , C_{ox} , and other familiar transistor quantities are not among the parameters we get to specify. In fact, apart from the terminal connections, it looks like we only get to choose W and L .

Is that all there is to it? Is circuit design just a matter of deciding how big each transistor is? Well, yes and no. With the exception of some advanced options (such as source or drain sharing, or multi-fingered gates), W and L really are the only transistor characteristics that you get to explicitly specify. You hook them together, size them correctly, and you almost have the whole thing. Really!

One possible design method, then, might be to just use W and L directly as design variables. This process would be something like the following:

1. Assume you have a usable topology.
2. Guess a bunch of values for W 's and L 's (and possibly R 's and C 's).
3. Simulate in Spice.
4. See if the design meets all the specifications
5. If not, modify W 's and L 's (and possibly R 's and C 's) and go back to Step 3.

Note: This kind of iterative process is sometimes called "Spice Monkeying," and its use is strongly discouraged. It is very common for designers of all experience levels to lapse into Spice Monkeying in the face of looming deadlines!

This strategy, however tempting, does not work well in practice. First, the sheer length of time required for simulation makes it impractical to run too many of them (and this is true even if you automate the process). More importantly a Spice-intensive design method is a blind trek, completely devoid of intuition. Spice is good for analyzing a design and making final tweaks, but it is not very good at helping you decide among the infinitude of topologies, sizings, and bias points available in an open-ended task.

All this is summarized in the simplified design flow shown in Fig. 5. The inner-most loop, loosely called the "hand calculations" phase, is where we have the best opportunities to make big-impact decisions. If we have meaningful and accurate small-signal models, we can make informed and confident decisions in this inner loop without resorting to frequent simulation. Let us see if we can develop a reliable link between small-signal models and actual transistor behavior.

0.4.1 Long-Channel Model Review

The best vehicle to carry us further on our quantitative discussion of MOSFET behavior is the long-channel model. Of course, I just told you that this model was inadequate, but, just to be clear, I am not advocating its complete abandonment. The derivation may be over-simplified, but it still usually gives the right kind of intuition; and we do not want to be robbed of that. In addition, it is simply a good place to start when discussing transistor modelling.

The long-channel model attempts to describe the relationships between drain-current, I_D , and the terminal voltages, V_{ds} and V_{gs} . The plot in Fig. 6 is one

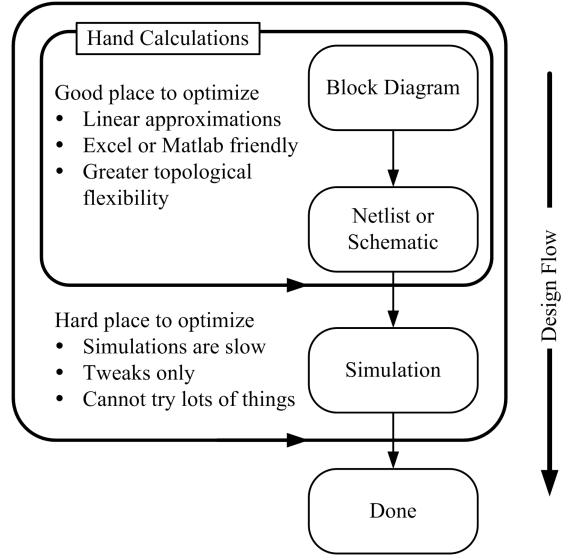


Figure 5: We can do more effective optimization in the hand-calculations phase than in the simulations phase of the design process.

of the most commonly used to display these relationships. One thing that is a little different about this plot, compared to others you may have seen, is the use of V_{ov} instead of V_{gs} . V_{ov} is called the overdrive voltage, and it is defined as follows:

$$V_{ov} = V_{gs} - V_T$$

V_{ov} tells you how *inverted* the channel is, and is a little easier to work with than V_{gs} , in part because it hides any dependence on V_T . *Inversion*, so-called because the material in the channel (e.g. p-type), starts to behave like the inverse type of material (e.g. n-type), can be roughly interpreted as "ON-ness." It is because it controls the level of inversion that we can consider V_{ov} to be a biasing variable. Sometimes the condition of having a very small V_{ov} is referred to as *weak inversion* while a large V_{ov} may cause *strong inversion*.

Also denoted in Fig. 6 are the three operating regions: cutoff, linear and saturation. We will quickly go over each one.

Note: This is an N-Channel-centric review. You will have to apply the usual flips to get the P-Channel relationships.

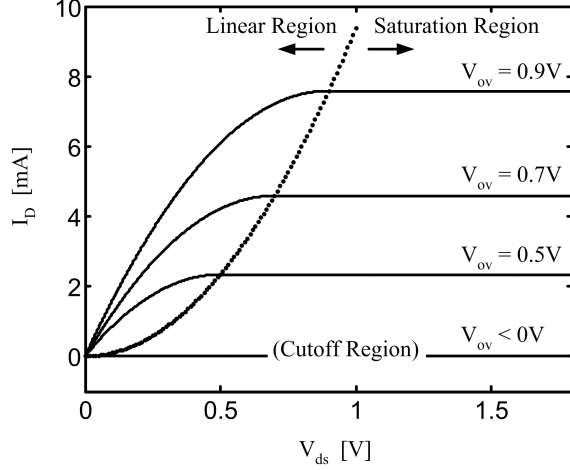


Figure 6: In the saturation region, I_D is primarily a function of V_{gs} .

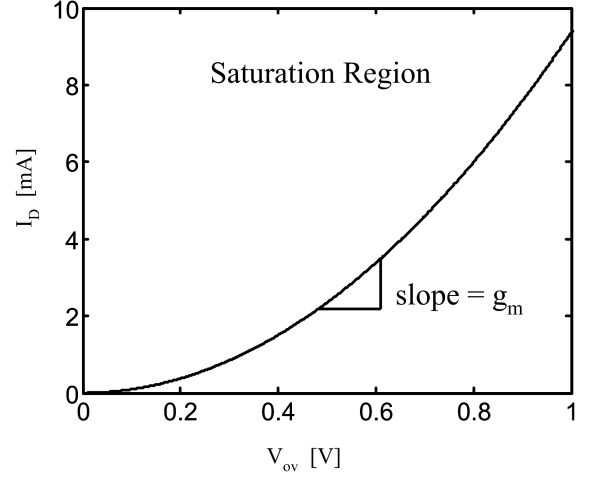


Figure 7: A saturation-centric view of transistor biasing.

Cutoff Region

Condition $V_{ov} < 0$, (or, equivalently, $V_{gs} < V_T$)
There is no channel inversion, so no current flows

$$I_D = 0$$

Linear Region

Condition $V_{ov} \geq 0$ and $V_{ds} < V_{dsat}$
There is channel inversion, but I_D is heavily affected by V_{ds}

$$I_D = \frac{1}{2} \mu C_{ox} \frac{W}{L} [2V_{ov}V_{DS} - V_{DS}^2]$$

The linear region is always of concern when the transistor is being used as a switch. In that case, you can also define its ON-resistance.

$$R_{on} \approx \frac{\partial V_{ds}}{\partial I_D}$$

Saturation Region

Condition $V_{ov} \geq 0$ and $V_{ds} \geq V_{dsat}$
 I_D becomes purely a function of the gate voltage, V_{ov} (not of V_{ds})

$$I_D = \frac{1}{2} \mu C_{ox} \frac{W}{L} V_{ov}^2$$

Saturation is the desired operating region for most of the transistors in the signal path, other than switches. In fact, for the rest of this article, we will operate almost exclusively in the saturation region. As long as each transistor has enough *headroom*, meaning that we maintain $V_{ds} \geq V_{dsat}$, then we can adopt a saturation-centric point of view, which is shown in Fig. 7. Note that, as V_{ov} increases, not only does I_D increase, but g_m grows as well due to the quadratic equation. In other words, g_m is a function of V_{ov} . Keep this picture in the back of your mind.

0.4.2 Introduction to the V_{ov} -Based Design Methodology

g_m , f_T and Making Sense of Transistor-Level Design

Now that we have been introduced to V_{ov} , we can develop it into a design variable. Remember, we eventually need it to tie into the small signal model shown in Fig. 2. The first element in the model we will work on is g_m , which is just the slope of the I_D vs. V_{ov} curve.

$$g_m = \frac{\partial I_D}{\partial V_{ov}} = \mu C_{ox} \frac{W}{L} V_{ov}$$

With a little algebraic manipulation, we can derive an interesting equation, which, as you may recognize, contains both of the biasing variables that we are investigating.

$$\frac{g_m}{I_D} = \frac{2}{V_{ov}}$$

I bring this up now for two reasons. First, it gives you an idea of the relationship between g_m/I_D and V_{ov} . We will see later that this equation is inadequate, because the actual relationship between these terms is much more complex. But for now, just keep in mind that these two quantities are somewhat similar.

A second reason for bringing up this equation is to define *transconductor efficiency*, which is just another word for g_m/I_D . This title is appropriate if you understand its meaning. I like to give it the intuitive units of mS/mA (rather than simplifying to $1/V$) because it captures the spirit of the term. It tells you how much g_m (in mS) you get when you invest a given I_D (in mA).

Going back to g_m itself, let us look at some expressions we can derive for it and see if we can make any sense out of them. Here is what we know so far:

$$g_m = \mu C_{ox} \frac{W}{L} V_{ov} = \frac{2I_D}{V_{ov}}$$

What does this mean? Is g_m is proportional to I_D ? If you want more transconductance, do you have to invest more current? That seems reasonable. But how about V_{ov} ? Well now I know we are in trouble because it is in the numerator in one case and in the denominator in the other!

In order to move forward, let us assume that we can program V_{ov} to any constant we want. Furthermore, we are going to think of V_{ov} as a *knob* that we can use to determine transistor behavior. We could have chosen a different knob, like I_D or W/L , but V_{ov} will turn out to be a better knob than these.

Let us do an example to show why a constant V_{ov} is a nice thing. In Fig. 8, we will operate M1 with a V_{ov} of 300mV. This is a reasonable value, and it is also an arbitrary choice. Also, arbitrarily, suppose that $I_D = 1mA$. How can we analyze this circuit?

M1 has 1mA of drain current, and superimposed on this is some signal current, i_s , which is a function of v_{in} . What kind of function? We are assuming a linear one based on g_m . In fact, we expect something like this:

$$v_{out} = -i_s \cdot R = -v_{in} \cdot g_m \cdot 1k\Omega$$

And because we know V_{ov} and I_D , we know g_m

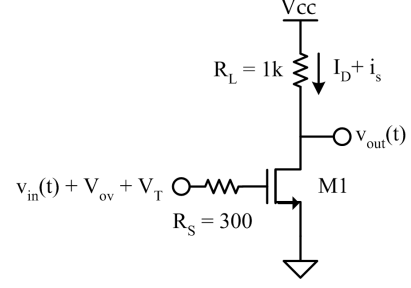


Figure 8: Example circuit.

$$g_m = 2I_D/V_{ov} = \frac{2 \cdot 1mA}{300mV} = 6.7 \frac{mA}{V}$$

$$\frac{v_{out}}{v_{in}} = -6.7 \frac{mA}{V} \cdot 1k\Omega = -6.7 \frac{V}{V}$$

In fact, we can use I_D to make the gain whatever we want. Suppose we double it.

$$g_m = 2I_D/V_{ov} = \frac{2 \cdot 2mA}{300mV} = 13.4 \frac{mA}{V}$$

$$\frac{v_{out}}{v_{in}} = -13.4 \frac{mA}{V} \cdot 1k\Omega = -13.4 \frac{V}{V}$$

What a breeze! Once we make V_{ov} a constant, we can program the gain by adjusting I_D . Analog design in CMOS is so easy.

Well, not quite. Though later when we get finished with g_m/I_D -based design, you might actually feel this way a little bit! But we are not there yet. There is something very obviously wrong with what we have done. You may have caught it if you are paying attention. If we can make V_{ov} whatever we want, then why not make it zero? Is that not the ideal value? I mean, if the following relationship is true,

$$g_m = \frac{2I_d}{V_{ov}}$$

then zero V_{ov} would make the circuit *infinitely* efficient! Certainly your suspicions are aroused, as they should be any time you manage to make a circuit infinitely efficient. We must be doing something wrong, but what?

It is actually something very basic. We have been ignoring speed. How fast is our transistor? Can we

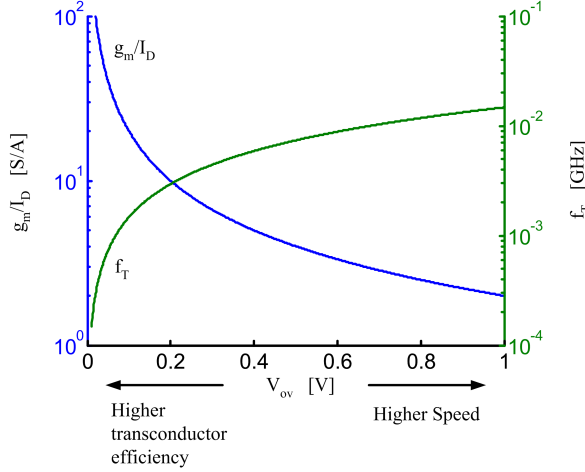


Figure 9: V_{ov} controls the tradeoff between transconductor efficiency and f_T .

put speed in terms of V_{ov} ? The answer, of course, is yes, and the derivation is painless. First, we define

$$f_T = \left(\frac{1}{2\pi} \right) \frac{g_m}{C_{gs}}$$

This is often called the *transit frequency*, and it is the answer to the question "How fast is this transistor?" In saturation, $C_{gs} = \frac{2}{3}C_{ox}WL$. If we plug that back into the long-channel model, we get

$$f_T = \frac{1}{2\pi} \frac{g_m}{C_{gs}} = \frac{1}{2\pi} \frac{2\mu V_{ov}}{3L^2}$$

Well, guess what. Now we know why we cannot make V_{ov} arbitrarily small – because it limits speed. We have found a g_m vs. f_T tradeoff, which is illustrated in Fig. 9.

Let us examine Fig. 9 more closely. Earlier, we decided that V_{ov} was a knob that we could use to adjust transistors. At the time, I told you it was an arbitrary decision, but now perhaps it is starting to look like it was also a good one. Two things that we care about, g_m and f_T (which could roughly translate as gain and bandwidth), are both dependent on V_{ov} in very simple ways. Even better, these two things have conflicting interests regarding V_{ov} , which means we can find an optimum and *make design choices*. For example, if we can live with a slow design, then we can use a low V_{ov} , which will yield a high transconductor efficiency (i.e. low power). On the other hand, if the circuit needs to be fast, we must operate with a

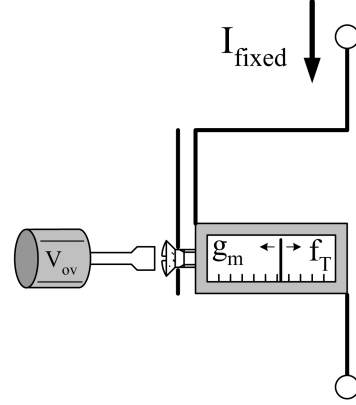


Figure 10: V_{ov} is like a g_m vs. f_T knob.

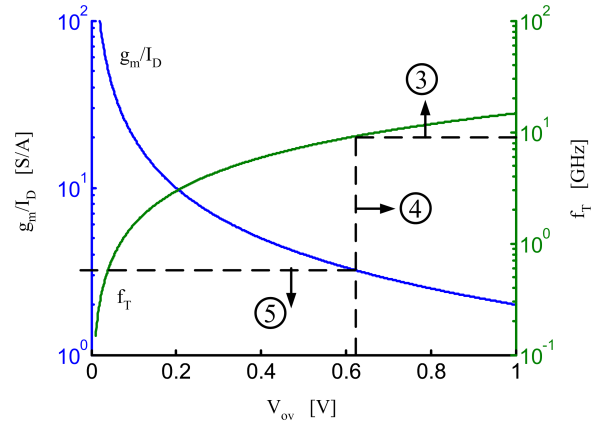


Figure 11: Design example showing how to use V_{ov} .

high V_{ov} and live with lower transconductor efficiency (i.e. high power).

This really is the heart of the matter. V_{ov} is useful precisely because it lets you manage the tradeoff between two of the things you need most in analog design. Think of it like the screwdriver in Fig. 10. For a fixed current, I_D , we can use V_{ov} to decide whether we want to spend that current investment on g_m (to get more gain) or on f_T (to get more bandwidth).

Now we want to plug all this back into the design flow. Remember Fig. 2. We want to be able to approximate a transistor, which is nonlinear, with a small signal model, which we can use in Signals-and-Systems analysis. We can illustrate how to do this with an example.

Using the circuit in Fig. 8, suppose we want 500MHz bandwidth and we want a gain of 10. One possible design flow, listed here, can be followed along in Fig. 11.

1. To achieve the required gain, we must have

$$g_m = 10 \frac{V}{V} / 1k = 10mA/V$$

2. And for an input pole at 500 MHz, we need

$$C_{gs} = \frac{1}{2\pi \times 300\Omega \times 500MHz} = 1.1pF$$

3. We can then easily calculate the required transit frequency

$$f_T = \frac{g_m}{C_{gs}} = \frac{10mS}{1.1pF} = 9.4GHz$$

4. Which means (see Fig. 9, Note 4)

$$V_{ov} \geq 75mV$$

5. Which means (see Fig. 9, Note 5)

$$g_m/I_D \leq 26mS/mA$$

6. And finally

$$I_D = \frac{g_m}{g_m/I_D} = \frac{10mS}{26mS/mA} = 385\mu A$$

Pretty easy, really. Not only that, but we know that our design is efficient. A larger V_{ov} would deliver a faster-than-necessary transistor, and we would waste power. A smaller V_{ov} would deliver a slower-than-required transistor, and we would not achieve the design goals.

Note that r_o has *not* been determined, so technically the small signal model from Fig. 2 is a little lacking. In this article, will not deal with r_o in a V_{ov} context. Later, once we have introduced g_m/I_D -based design, we will investigate the limitations due to r_o .

In summary, finding an optimum operating point using the V_{ov} -based design methodology is much more straightforward than working brute force with W or L . And thinking in terms of V_{ov} makes it easy to deal with the interdependencies of g_m , C_{gs} and biasing than by trying to manage these independently. The only real problem, as we pointed out earlier, is that the long-channel model is not accurate.

0.4.3 The Limitations of V_{ov} -Based Design

We have been hinting all along that the long-channel model is just way too simple to make V_{ov} -based design

reliable - even if we patch the model with extra Greek letters. Now it is time to present some hard evidence.

The first bit of evidence is simple common sense: Why do you think Spice uses dozens of transistor parameters during simulation? Do you think that it ignores all but 2 or 3 of them? Of course not! Consequently, we should be shocked (and modelling engineers should be embarrassed) if μ and C_{ox} proved to be as complete as the entire Spice model.

Figs. 12 and 13 illustrate this graphically. In Fig. 12, we compare actual transconductor efficiency simulated in Spice to the long-channel prediction. For large values of V_{ov} , the long-channel model is only off by 25% - not too bad. But for small V_{ov} , the values are nonsense - most notably at $V_{ov} = 0$, where it still insists on infinite transconductor efficiency. Fig. 13 shows that the long-channel model does no better in predicting f_T . And the intersection where *both* graphs are on target is hardly existent at all.

You may also have noticed that Figs. 12 and 13 contain a region of *negative* values for V_{ov} , which is also called the subthreshold region. The long channel model predicts that for $V_{ov} \leq 0$, $I_D = 0$, implying we should never bias a MOSFET near V_T because it might turn off. But the simulation data in Fig. 12 shows that g_m/I_D actually continues to climb as we head into the subthreshold region, which means that this may really be a useful biasing point. Is it?

In fact, the subthreshold and weak inversion regions are very important in *low-power* designs. Fig. 13 explains the tradeoff, which is that subthreshold transistors are slow. But keep in mind that they are not *that* slow in newer technologies. In fact they can often be much faster than we need them to be, which allows designers to trade some of that speed for low power. For many of today's power-constrained designs, subthreshold operation is imperative, and a model that does not accomodate this region is useless.

What does all this mean? It means that, although V_{ov} is a very good design variable in theory, it does not work in practice. If the long channel model were accurate, then V_{ov} -based design would work brilliantly and there would be no reason to look elsewhere. But the long-channel model does not work and so we need a new design variable: something in the spirit of V_{ov} , but that yields better agreement between hand calculations simulation.

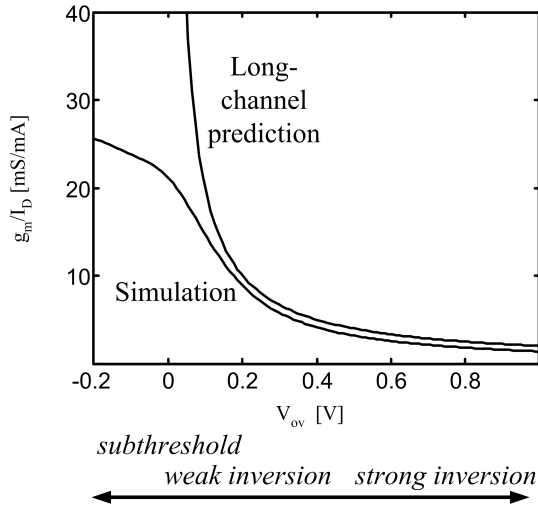


Figure 12: V_{ov} does not predict transconductor efficiency very well.

0.4.4 Introduction to the g_m/I_D -Based Design Methodology

g_m/I_D as a Design Variable

For starters, suppose we tried to keep V_{ov} as a design variable. We could use simulations to see how V_{ov} affects both transconductor efficiency and f_T , similar to what was done in Figs. 12 and 13. Using those two charts, we can certainly find the best value for V_{ov} to suit our needs.

Fortunately, someone has already come up with a much better idea that encapsulates all this information compactly. Recall that V_{ov} and g_m/I_D are similar, biasing-related quantities. If they are so similar, maybe we can get rid of one of them. The clever trick is to plot f_T vs. g_m/I_D directly, as shown in Fig. 14. This plot lets you see exactly how increasing transconductor efficiency comes at the cost of f_T . We are cutting out the V_{ov} middle-man, so to speak. From here on, g_m/I_D is the only biasing variable we will need.

By the way, where is the subthreshold region in Fig. 14? You can get a hint by looking at Fig. 12. The subthreshold region is where we expect to find the highest values for transconductor efficiency. In other words, it lies somewhere on the far *right* side of Fig. 14. Does it matter exactly where it begins? Of course not! We know g_m/I_D . We know f_T . The region we are in is irrelevant. In fact, with the exception of the linear region, of which we must always be care-

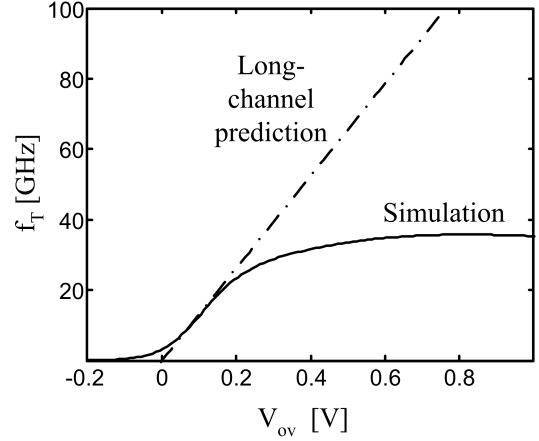


Figure 13: V_{ov} does not predict f_T very well.

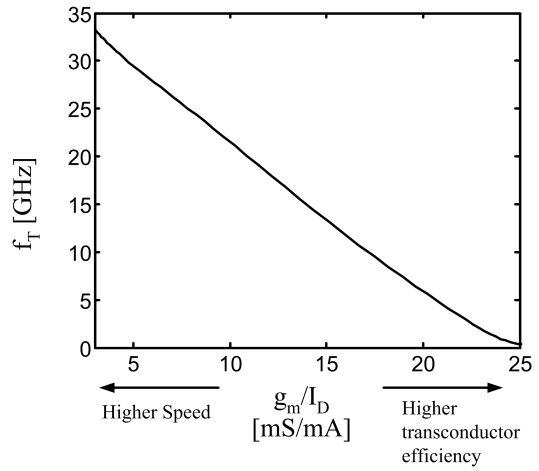


Figure 14: A more direct depiction of the g_m/I_D vs. f_T tradeoff.

ful, all the regions of operation become transparent when we use the g_m/I_D -based design methodology.

Again, let us do an example based on the circuit in Fig. 8. And, again, assume that we want a signal bandwidth of 600MHz , a gain of 10, and that we want to use as little power as possible. We can follow along in Fig. 15 to do the design.

1. To achieve the required gain, we again must have

$$g_m = 10 \frac{V}{V} / 1k = 10\text{mA}/V$$

2. And for an input pole at 500MHz , we still need

$$C_{gs} = \frac{1}{2\pi \times 300\Omega \times 500\text{MHz}} = 1.1\text{pF}$$

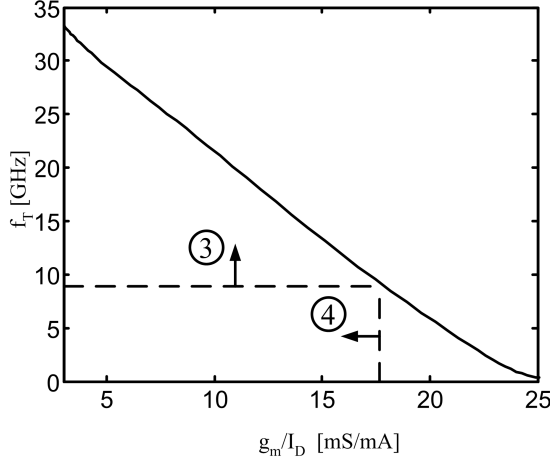


Figure 15: Using g_m/I_D in a simple design.

3. So the transit frequency must be

$$f_T = \frac{g_m}{C_{gs}} = \frac{10 \text{ mS}}{1.1 \text{ pF}} = 9.4 \text{ GHz}$$

4. Which means (see Fig. 15, Note 4)

$$g_m/I_D \leq 17.5 \text{ mS/mA}$$

5. And finally

$$I_D = \frac{g_m}{g_m/I_D} = \frac{10 \text{ mS}}{17.5 \text{ mS/mA}} = 570 \mu\text{A}$$

Easy. Of course, so was the V_{ov} -based method we looked at earlier. The important difference is that these new relationships are *accurate*.

Fig. 16 is a more complete (and actually our final) version of the f_T vs. g_m/I_D design chart. It incorporates the effect of channel length, L , on transistor speed. Several lengths are included, ranging from the process minimum up to a reasonably large value. We will explain how to create this chart in a later section.

Let us look closely at Fig. 16. For any given value of g_m/I_D , a larger L *always* means a slower transistor. This means that, if we have no other constraints, we should always choose the shortest-length transistor available (i.e. the process minimum). It also means that there must be some other constraint we might need to consider. That constraint is r_o , and we will cover it now.

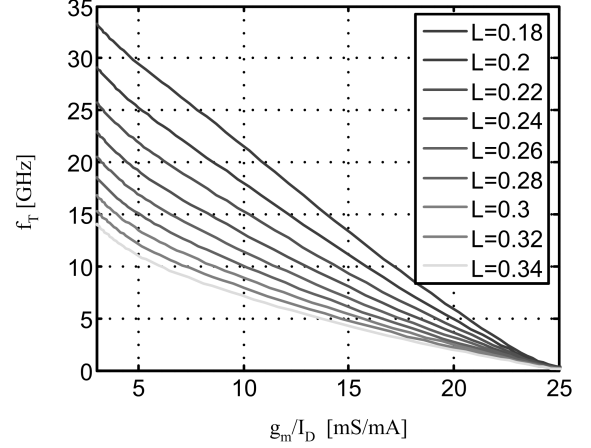


Figure 16: An f_T design chart for a $0.18 \mu\text{m}$ process.

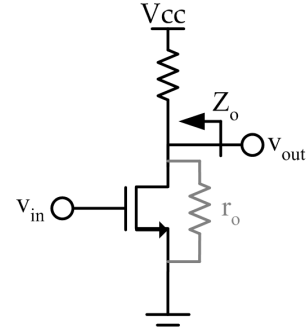


Figure 17: r_o of a common-source amplifier appears as an additional load.

What about r_o ?

Fig. 17 explicitly highlights r_o 's place in a transistor circuit. As you know, r_o is simply another load in parallel with R_L . In practice, we usually either want $r_o \gg R_L$ so that r_o can be ignored, or we will make R_L very large (perhaps using a current source instead of an actual resistor) in order to get as much gain as we can. I want to examine the second case. Let us see what happens as $R_L \rightarrow \infty$.

Fig. 18 illustrates this case effectively. When r_o becomes the dominant resistive load, the overall gain is limited by what we refer to as the *intrinsic gain* of the transistor. Intrinsic gain is the product of g_m and r_o and tells us the highest (Voltage) gain we can possibly get.

$$\text{Intrinsic Gain} = g_m r_o \left(\frac{V}{V} \right)$$

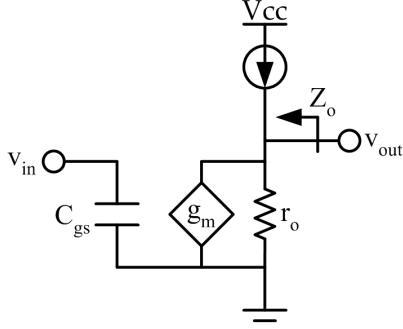


Figure 18: What is the maximum gain we can get from a transistor?

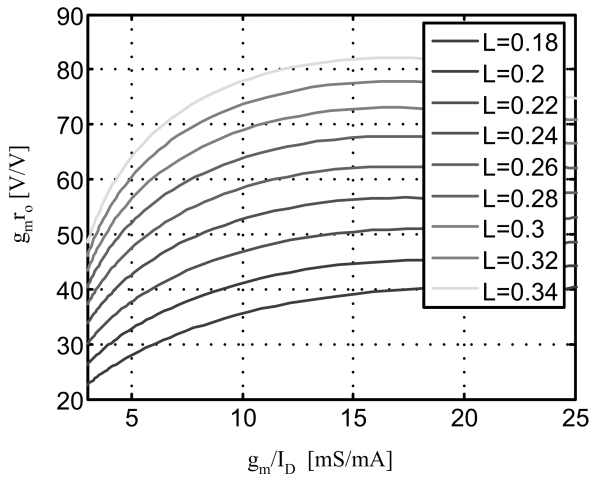


Figure 19: An intrinsic gain chart for a 0.18 μm process.

In practice, intrinsic gain is a more convenient number to know than r_o itself (we will see why shortly). Mathematically they are quasi-equivalent. That is, you can put *gains* "in parallel" just like you can put R 's in parallel. For example, if a circuit has an ideal gain of 10, and you use a transistor with an intrinsic gain of 100, then the net gain degrades by about 10%.

We can use the concept of intrinsic gain to create another design chart. Fig. 19 is similar to Fig. 16, except that intrinsic gain is the dependent variable. Like Fig. 16, it is plotted vs. g_m/I_D , and that is done for several different channel lengths.

Figs. 16 and 19, taken together, comprise a very powerful design tool. Not only are they extremely helpful in transistor implementation, but they also allow a designer to understand the capabilities of the transistors available in the technology target.

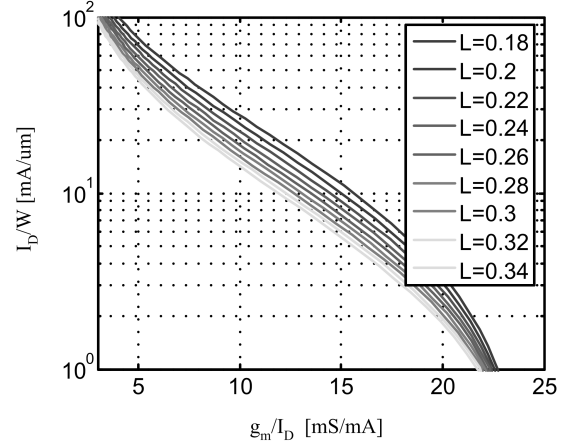


Figure 20: A biasing chart for a 0.18 μm process.

As an example, suppose you would like a circuit with a gain of at least 50. A quick glance at the intrinsic gain chart tells you that this is certainly feasible. Choosing the minimum channel length, $L = 0.18\mu\text{m}$, is probably too risky since it leaves no margin, but perhaps $L = 0.28\mu\text{m}$ would be conservative enough.

Now, what if we want a gain of 100? This time a quick glance at the chart says that we cannot get this from a single transistor. We might need to either use multiple stages, or perhaps try a cascode circuit in order to increase the gain. Those are problems we can solve. The important thing is that we know this stuff *now*. We do not have to waste time trying to make the design work with a single transistor, only to find out through several Spice iterations that it cannot be done.

In summary, these two charts give a fairly complete picture of transistor behavior based on g_m/I_D as a design variable. We know how both f_T and $g_m r_o$ are affected by both g_m/I_D and L . Because of this we can choose the best L and g_m/I_D for the job. And because the charts are simulation-based, we are confident that they are accurate.

A small warning is in order before we move on. Be aware that r_o is very dependent on V_{ds} . The values presented in the charts are for $V_{ds} = V_{DD}/2$. If V_{ds} starts to drop to near V_{dsat} (in other words, if you do not have much headroom), then the intrinsic gain may drop substantially (perhaps as much as 4 to 5 $\times$).

Biasing Using g_m/I_D

There is one final chart that I want to introduce. Figs. 16 and 19 are *design* charts. This new chart, shown in Fig. 20, is a *biasing* chart. You use Figs. 16 and 19 to decide values for L and g_m/I_D . We can then easily use Fig. 20 to look up the value of W required to bias the transistor at our desired g_m/I_D value. W ? That's right! By the time we get to this chart, everything else is determined. We know L from gain requirements. We picked g_m/I_D in order to guarantee some particular f_T . And I_D is picked such that we get the right value for g_m . The only thing left is W .

0.4.5 A Deeper Understanding of the g_m/I_D Method

What is the *magic* of the g_m/I_D -based design methodology? For example, why does a specific value of g_m/I_D always result in the same f_T , no matter what else you do? Doesn't that seem strange? While understanding this is not critical to using the methodology, it is worthwhile to have an intuitive idea of why it works. A simple thought experiment actually makes this rather easy.

Fig. 21 shows a progression of transistor circuits. Suppose we measure both g_m and C_{gs} for the transistor in circuit *a*. Logically, because the terminal voltages and bias currents remain unchanged, each of the transistors in circuit *b* will also have these same values for g_m and C_{gs} . But, since there are two of them working in parallel, the composite behavior presents a doubling of I_D , g_m , and C_{gs} . The key is that, because all of three of those things scaled *together*, circuit *b* as a whole has exactly the same g_m -to- I_D and g_m -to- C_{gs} (i.e. f_T) ratios as circuit *a*. In fact, no matter how many we put in parallel, we will always get the same g_m/I_D and f_T . Do not move on until you convince yourself that this is true.

Next, we need to make the step from circuit *b* to circuit *c*. I actually think that you should find this quite natural. For example in a current mirror, we expect that we can use width ratios to create specific current ratios. In fact sometimes these ratios are implemented with parallel unit elements anyway.

Finally, to complete the thought experiment, we must agree that we can generalize this idea to say that *any* ratio will work, not just integers. This generalization means that g_m , I_D and C_{gs} all scale linearly with W . If W increases by 25%, then we expect g_m , I_D and

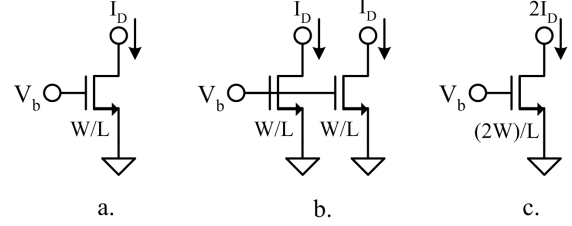


Figure 21: A progression of transistor circuits that all have the same value of g_m/I_D and f_T .

C_{gs} to all increase by 25%, which always maintains the same *ratios* of g_m -to- I_D and g_m -to- C_{gs} .

Intrinsic gain scales in the same way. The composite behavior of circuit *b* presents twice the g_m of circuit *a*, but it also puts two r_o 's in parallel. You get twice the g_m but half the r_o , resulting in a composite intrinsic gain that is exactly the same as the transistor in *a*. No matter how many we put in parallel, the intrinsic gain will be the same. So, like f_T , if we plot intrinsic gain as a function of the ratio g_m/I_D , then it becomes independent of W . It is this width-independence that makes intrinsic gain preferable to r_o in terms of design convenience.

This is the way to think about the g_m/I_D -based design methodology. We basically characterize a single transistor of width W . For this one device, we sweep the gate voltage and measure the resulting values for g_m , I_D , C_{gs} and r_o . Once we know the relationships between these parameters for the transistor of width W , we can rely on linear scaling to determine the behavior of a transistor of width αW .

That is all there is to it. As long as everything scales with W , then the g_m/I_D methodology will hold. Of course, we also know that these ratios are not perfect. Two transistors in parallel of width W do not perform *exactly* the same as one transistor of width $2W$, but they are within a few percent. Remember Fig. 4: we only expect to get within 10-20% anyway, because the final tweaks will be done in Spice. And this is much closer than we can get trying to rely on the long-channel model.

0.5 A Top-to-Bottom Design Example

In the previous two sections, we have developed the g_m/I_D -based design methodology. Now I want to do

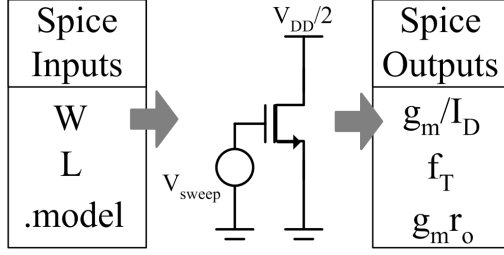


Figure 22: Method of making design charts.

a complete design example so that you can see exactly how it is used.

0.5.1 Problem Description

You are tasked with creating a differential amplifier in a $0.18\mu m$ process. The amplifier must have the following characteristics:

1. Gain of ≈ 10
2. Bandwidth of $\approx 200MHz$
3. Drive a $1pF$ load
4. Be driven by a 300Ω source
5. Lowest possible power

0.5.2 Characterize the Technology

The first step is to characterize the technology target, which just means that we need to create some design and biasing charts (Note that, in this case, Figs. 16, 19, and 20 were generated based on this technology target, so we will simply refer to them). The circuit that you will simulate in order to create these charts is shown in Fig. 22.

In order to fill in the curves, you need to sweep the voltage source while monitoring the transistor, and you need to repeat that sweep for a variety of channel lengths. Feel free to use the following (Hspice) lines to access the necessary 'internal' transistor characteristics.

```
.probe gmID = par('gmo(m1)/i(m1)')
.probe fT = par('gmo(m1)/(2*3.14*cggbo(m1))')
.probe gmro = par('gmo(m1)/gdso(m1)')
.probe idw = par('i(m1)/w(m1)')
```

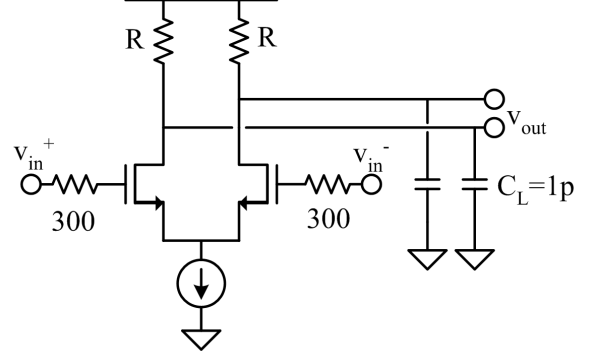


Figure 23: Potential topology for our design example.

Using a plotting tool such as Matlab, you can then read all these variables from the `filename.SW0` file that contains all the sweep data and create plots similar to Figs. 16, 19, and 20. Also, it will benefit you to consult these charts as we go along in order to get used to picking off the values.

0.5.3 Choose a Topology

Fig. 23 pictures a potential circuit that we can use for this design. Will this topology work? We can be pretty confident that it will. According to Fig. 19, we might need a longer-than-minimum-length channel and a g_m/I_D of at least 10 in order to make sure that intrinsic gain (which isn't extremely reliable) is not the dominant gain determinant, but even under these conditions, Fig. 16 shows that we can still achieve high f_T .

Actually, the very first thing we can get out of the way is L , because we have hard constraints on intrinsic gain. For example, we can choose $L = 0.22\mu m$, which will keep the intrinsic gain around 50 (for moderate values of g_m/I_D), meaning that it will only have a 20% effect on overall gain.

$$L = 0.22\mu m$$

The next thing we can calculate is a value for R . This is a straightforward Signals-and-Systems calculation. Each resistor forms a pole with the $1pF$ capacitive load, and we want that pole at $200MHz$.

$$R = \frac{1}{2\pi \times C \times 200MHz} = 800\Omega$$

and knowing R , we can calculate the g_m required to achieve a gain of 10.

$$g_m = \frac{10}{R} = 12.5 \text{ mS}$$

Unfortunately, the 200 MHz pole is not the only one in our system. Each 300Ω input resistor forms a secondary pole with C_{gs} , which can complicate the frequency response if it lies in the vicinity of the 200 MHz pole. In order to keep this secondary pole from affecting the frequency response, we can push it to a higher frequency. For example, a $10\times$ margin beyond the dominant pole should allow our circuit to maintain approximate single-pole behavior. This sets the value for C_{gs} .

$$C_{gs} = \frac{1}{2\pi \times 300 \Omega \times 2 \text{ GHz}} = 265 \text{ fF}$$

Knowing g_m and C_{gs} , we can calculate the transit frequency

$$f_T = \frac{1}{2\pi} \frac{g_m}{C_{gs}} = \frac{1}{2\pi} \frac{12.5 \text{ mS}}{265 \text{ fF}} = 7.5 \text{ GHz}$$

Now that we know L and f_T , we have fixed the value for g_m/I_D . This can simply be read off the f_T design chart.

$$g_m/I_D = 16.5 \text{ mS/mA}$$

And since we know both g_m and g_m/I_D , we can determine I_D

$$I_D = \frac{g_m}{g_m/I_D} = \frac{12.5}{16.5} \approx 0.76 \text{ mA}$$

Of course, our current source will need to deliver double this current because we need to power two transistors. Finally, the very last step is to determine the value of W that ensures that we operate at our desired transconductor efficiency of 16.5. According to the biasing chart, a transconductor efficiency of 16.5 mS/mA and a length of $0.22 \mu\text{m}$ corresponds to a current density of $6.5 \mu\text{A}/\mu\text{m}$. So we can calculate W .

$$W = \frac{0.76 \text{ mA}}{6.5 \frac{\mu\text{A}}{\mu\text{m}}} = 117 \mu\text{m}$$

The final design is shown in Fig. 24, and a Frequency response plot is shown in Fig. 25. How do you think

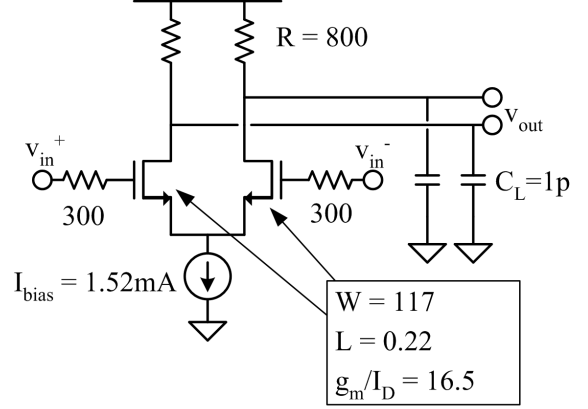


Figure 24: Final version of our design example.

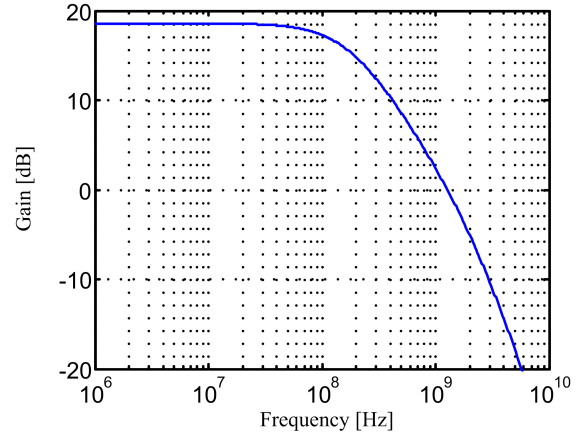


Figure 25: Performance of our design example.

we did? The gain is a little low. We were shooting for 10 (20dB), but we only achieved about 8.5. Also, we did not quite hit 200MHz bandwidth. What happened?

First of all, both shortcomings can be explained easily. We expected to underachieve in gain because we did not account for finite r_o in the hand calculations. Remember we were working with an intrinsic gain of only about 50. We could have been a little more conservative and maybe tried to overshoot the gain target by 20% or so. The shortcoming in bandwidth is easy to understand as well. First of all, C_L is not the only capacitive load we have to drive! In parallel with C_L is the transistor itself, which of course has some drain capacitance. In addition, the secondary pole is only a factor of 10 above this pole. This makes its effect small, but still noticeable.

The g_m/I_D -based design methodology actually did very well. We know a few things that we might want to take into account if we performed another design iteration (we could estimate drain capacitance based on W and include intrinsic gain limitations for starters). But the methodology did correctly predict the things that we should expect. For example, from the simulation output file, we can read off the actual values for g_m and C_{gs} calculated by Hspice.

```
gm 12.6378m
cgtot 264.2388f
```

As you can see, these are extremely close to the values we requested. In other words, we do not have anything to complain about regarding the methodology! The fact that we did not meet the design goals is simply our own fault. The methodology cannot make up for obvious oversights in the design process. We still need to consider things like Miller capacitance, drain capacitance, feedthrough, etc. But those are all things we can account for in Signals-and-Systems analyses. And now with the help of the g_m/I_D -based methodology, we can actually be confident that the small signal parameters that we work with in Signals-and-Systems analyses will be correctly realized in the final design.

Finally, there are more sophisticated approaches you may try. I have been presenting the g_m/I_D design data in the form of charts, but there is no reason why you could not import them as tables into your favorite mathematics package. Once you can programmatically retrieve these data from lookup tables, you are free to employ all manner of design procedures. You could even wrap an optimization engine around everything in order to maximize some particular performance parameter.

0.6 Conclusion

In conclusion, the g_m/I_D -based design methodology is the best tool we have for linking small signal values, such as g_m and f_T , to physical parameters such as W , L , and V_{gs} . It encapsulate the g_m vs. f_T trade-offs compactly and predicts simulated performance very accurately. In addition, it gives the designer an idea of the limitations of the technology target, which helps drive architectural decisions early in the design cycle. Finally, the data can be imported as tables into mathematics packages so that designers can use sophisticated optimization routines, giving it all the advantages of an equation-based approach, but yield-

ing much more accurate results.